

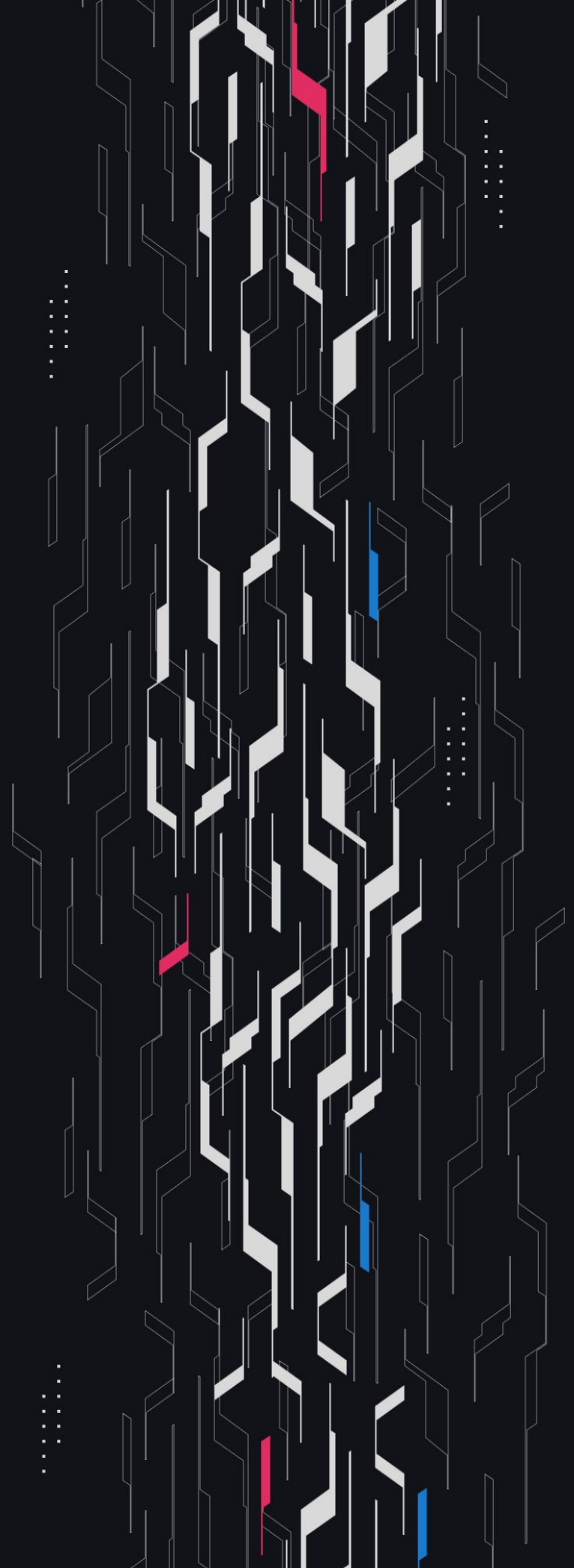
GA GUARDIAN

USDT0

USDT0 Connector

Security Assessment

February 28th, 2026



Summary

Audit Firm Guardian

Prepared By Minato Namakazi, Osman Ozdemir

Client Firm USDT0

Final Report Date February 28, 2026

Audit Summary

USDT0 engaged Guardian to review the security of their USDT0 Connector. From the 24th of November 2025 to the 7th of January 2026, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 5 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the remediation review have not been reviewed by Guardian.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 19

About Guardian 20

Project Overview

Project Summary

Project Name	USDT0
Language	Solidity
Codebase	https://github.com/Everdawn-Labs/usdt0-ofc-contracts https://github.com/hiero-ledger/hiero-consensus-node
Commit(s)	Hiero Main Review commit: b71f1a623f0144d486adaa99619421e86ea05153 Hiero Remediation Review commit: c6469486b342b8a2e386d0e5b46702dd0e35f816 OFT-Contracts commit: c1c12b2bcd33ec695066227b7736ba80527e242e

Audit Summary

Delivery Date	February 28, 2026
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	0	0	0	0	0	0
● Low	1	0	0	0	0	1
● Info	7	2	0	5	0	0

Audit Scope & Methodology

In Scope:

hedera-node/hedera-token-service/src/main/java/com/hedera/node/app/service/token/*

hedera-node/hedera-token-service-impl/src/main/java/com/hedera/node/app/service/token/impl/*

hedera-node/hedera-smart-contract-service/src/main/java/com/hedera/node/app/service/contract/*

hedera-node/hedera-smart-contract-service-impl/src/main/java/com/hedera/node/app/service/contract/impl/*

hedera-node/hedera-util-service/src/main/java/com/hedera/node/app/service/util/*

hedera-node/hedera-util-service-impl/src/main/java/com/hedera/node/app/service/util/impl/*

Code Metrics

Java:

- Files: 588
- Blank Lines: 7,014
- Comment Lines: 17,085
- Code Lines: 50,784

Total:

- Files: 588
- Blank Lines: 7,014
- Comment Lines: 17,085
- Code Lines: 50,784

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Locked Initial Supply In HTSConnectorUpgradeable	Validation	● Low	Resolved
I-01	Missing Validation Of Key Lengths	Validation	● Info	Acknowledged
I-02	Warning About New HTS Precompiles	Informational	● Info	Acknowledged
I-03	Conflicting transferToken Documentation	Documentation	● Info	Acknowledged
I-04	Zero Amount Operations Are Not Rejected Early	Validation	● Info	Acknowledged
I-05	contract_Id Keys Break Freeze And Wipe	Warning	● Info	Pending
I-06	Token Keys Cannot Be Updated Via Connector	Superfluous Code	● Info	Acknowledged

L-01 | Locked Initial Supply In HTSConnectorUpgradeable

Category	Severity	Location	Status
Validation	● Low	HTSConnectorUpgradeable.sol: 135C26-135C52	Resolved

Description

The `createHTSToken` mints the `initialTotalSupply` to the treasury, which is the `HTSConnector` contract. There is no function to transfer this initial supply out from the treasury.

All transfers in the contract assume mint-then-transfer for new supply or pull-then-burn, but nothing can move the initial treasury balance. If `_tokenInitialTotalSupply > 0`, these tokens are effectively stranded.

Recommendation

While `initialTotalSupply` is set to 0 for mainnet in the deployment script, it is advisable to also enforce at the contract level that the `HTSConnectorUpgradeable` has no initial supply that would be locked.

Resolution

USDT0 Team: Resolved.

I-01 | Missing Validation Of Key Lengths

Category	Severity	Location	Status
Validation	● Info	KeyHelperUpgradeable.sol: 130C1-131C44	Acknowledged

Description

getKeyValueType in KeyHelperUpgradeable contract accepts arbitrary bytes for ED25519 and SECP256K1 keys without validating length.

HTS expects ED25519 public keys to be 32 bytes and compressed SECP256K1 public keys to be 33 bytes. Passing an invalid length will cause HTS precompile calls to fail.

The function is only used by the current HTSConnectorUpgradeable during the createHTSToken call, where the \\$.adminKey is provided as a SECP256K1 key. However, _adminKey is set during initialization and can be assigned to a byte array of any size.

Recommendation

Consider adding length checks in the getKeyValueType function to ensure that all ED25519 keys are 32 bytes and all SECP256K1 keys are 33 bytes, or alternatively enforcing that _adminKey is 33 bytes during initialization.

Resolution

USDT0 Team: Acknowledged.

I-02 | Warning About New HTS Precompiles

Category	Severity	Location	Status
Informational	● Info	HederaTokenService.sol: 11	Acknowledged

Description

As introduced in [HIP-646](#), fungible tokens on Hedera can include metadata. While this was previously unsupported by smart contracts, a new proposal, [HIP-1028](#), enables metadata support for fungible tokens.

According to [HIP-1028](#), a new HTS precompile will be deployed at address 0x16c, while 0x167 will remain available for backward compatibility.

The current [HTSConnector](#) uses the precompile at 0x167 and does not support the metadata key or metadata features for fungible tokens. It is important to be aware of the breaking changes if USDT0 decides to use this metadata feature in the future.

Recommendation

Be aware of this situation.

Resolution

USDT0 Team: Acknowledged.

I-03 | Conflicting transferToken Documentation

Category	Severity	Location	Status
Documentation	● Info	IHederaTokenService.sol: 149-153	Acknowledged

Description

IHederaTokenService interface's transferToken documentation mentions "(positive amount) or receiving (negative amount)", while the parameter docs state "Non-negative value to send. a negative value will result in a failure".

Recommendation

Update the NatSpec comment on the IHederaTokenService interface.

Resolution

USDT0 Team: Acknowledged.

I-04 | Zero Amount Operations Are Not Rejected Early

Category	Severity	Location	Status
Validation	● Info	HTSConnectorUpgradeable.sol: 194-196	Acknowledged

Description

_debit, _credit, mintTokenPublic and burnTokenPublic function in the HTSConnectorUpgradeable accept _amountLD == 0 and only guard against values exceeding int64.max.

HTS burn/mint require positive non-zero amounts, so zero results revert in later precompile calls. This wastes gas and degrades UX without a clear error at the top of the call path.

Recommendation

Require _amountLD > 0 and revert early with an error.

Resolution

USDT0 Team: Acknowledged.

I-05 | contract_Id Keys Break Freeze And Wipe

Category	Severity	Location	Status
Warning	● Info	HTSConnectorUpgradeable.sol: 113-122	Pending

Description

On Hedera, a contractID key is not compatible with contracts that execute via delegatecall Hedera explicitly documents that contractID requires "the code in the active message frame to belong to the contract" whereas delegatableContractId exists specifically to support delegatecall.

Make sure that the HTS precompile will recognize the used wallet as satisfying a contractID key or change it if needed.

This is important for USDT0 because it could break the ability to respond to incidents, stolen funds, sanctions/compliance actions, or any emergencies.

Recommendation

If needed change HTS token creation to delegatableContractId is specifically for contracts that might execute logic via delegatecall.

Resolution

USDT0 Team: Pending.

I-06 | Token Keys Cannot Be Updated Via Connector

Category	Severity	Location	Status
Superfluous Code	● Info	HederaTokenService.sol: 139C1-144C6	Acknowledged

Description

HederaTokenService.sol defines an updateTokenKeys function. However, this function is marked as internal and is never invoked by HTSConnectorUpgradeable, so it cannot be called. As a result, updating token keys via precompile invocation is not possible, even if that was the original intent.

Recommendation

Either add an access-controlled external function that allows updating keys through the connector, or remove the updateTokenKeys internal function from HederaTokenService.sol.

Resolution

USDT0 Team: Acknowledged.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
I-01	Fix If I-08 Is Not In Final Commit	Informational	● Info	Pending

I-01 | Fix If I-08 Is Not In Final Commit

Category	Severity	Location	Status
Informational	● Info	HTSConnectorUpgradeable.sol: 113C1-120C26	Pending

Description

The comment in issue I-08 states that the issue was fixed in `acf07bcea340c1ca8ed039399c816adf8a851a33`. However, this fix is not reflected in the final commit. The freeze and wipe keys still use `CONTRACT_ID`.

Recommendation

No change is required if it was later decided to continue using `CONTRACT_ID` despite the comment indicating a fix. Otherwise, consider applying the fix to the latest commit.

Resolution

USDT0 Team: Pending.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>